OpenTelemetry Observability Lab

Metrics, traces and logs through one Collector: the concepts, the tools, and a hands-on lab you can run on your laptop

Inside this guide	Section
OpenTelemetry in one minute	1
Core concepts: signals, spans, SDK, OTLP	2
The OpenTelemetry Collector	3
Tech stack: what each tool does	4
Architecture, data flow and ports	5
How this helps in real production	6
Lab: step by step, with what you will see	7
Production vs this lab	8
Interview questions with short answers	9
Official references	10

Before you start. Everything runs locally with **Docker Compose**: no cloud account and no cost. You need Docker Desktop and about 4 GB of free memory. It takes about 1 to 2 hours.

Author: Aniruddha Choudhary, Cloud / DevOps / MLOps engineer. Tested November 2026: OpenTelemetry Collector contrib 0.164, OpenTelemetry Python SDK 1.45, Jaeger 2, Loki 3, Prometheus, Grafana. Code: companion GitHub repository opentelemetry-observability-lab.

1. OpenTelemetry in one minute

When a user says "*checkout is slow*", you need three answers:

- **Is something wrong?** Metrics: numbers over time.
- **Where is it slow or failing?** Traces: one request's path across services.
- **What exactly happened?** Logs: detailed messages.

OpenTelemetry (OTel) is the open standard for producing and sending all three. You instrument an app **once** and can send the data to **any** backend: Prometheus, Jaeger, Loki, Datadog, Grafana Cloud and more.

> **Think of a universal power adapter.** Your app plugs into OpenTelemetry once, and OpenTelemetry plugs into whichever monitoring tool you choose. Switching tools no longer means changing your code.

Prometheus vs OpenTelemetry

	Prometheus	OpenTelemetry
What it is	A metrics database + scraper + alert engine	A standard + SDKs + Collector for shipping telemetry
Signals	Metrics	Metrics, traces and logs
Stores data?	Yes	No. It sends data to a backend
Typical model	Pull (scrapes /metrics)	Mostly push (OTLP)

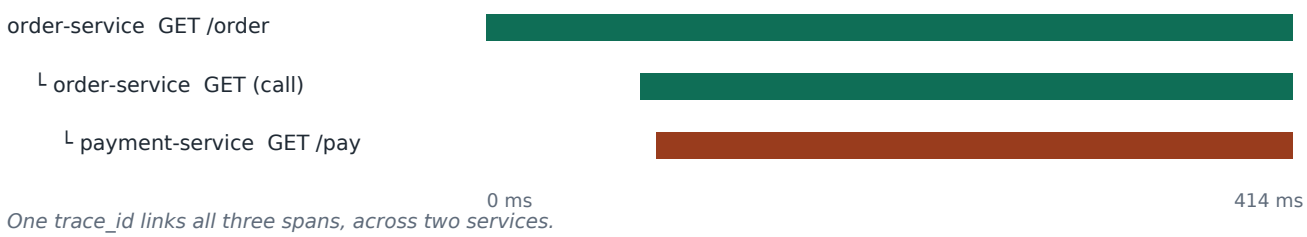
They work together: OpenTelemetry **collects and ships**, while Prometheus, Jaeger and Loki **store and query**.

2. Core concepts

2.1 The three signals

Signal	What it is	Answers	Backend in this lab
Metrics	Numbers over time	Is something wrong?	Prometheus
Traces	One request's journey, made of spans	Where is it slow or failing?	Jaeger
Logs	Timestamped messages	What exactly happened?	Loki

2.2 Traces and spans



One trace_id links all three spans, across two services.

- A **span** is one unit of work, with a start time, a duration and attributes (method, URL, status code).
- Spans form a **parent to child** tree, and all of them share one **trace_id**.
- **Context propagation:** when order-service calls payment-service, OTel adds a `traceparent` HTTP header, so the second service **joins the same trace**.

2.3 What gets installed in an app

Part	Role
API	The functions you would call in code (start_span, ...)
SDK	The engine: creates spans, metrics and logs, and batches them
Instrumentation libraries	Auto-hooks for frameworks: Flask, requests, logging
Exporter	Sends the data out (here over OTLP to the Collector)
opentelemetry-instrument	The launcher: starts the SDK, applies the hooks, then runs your app

2.4 Auto vs manual instrumentation

	Auto (this lab)	Manual
Code changes	None: run the app with opentelemetry-instrument	Import OTel and create spans and metrics yourself
You get	HTTP spans, HTTP metrics, logs with trace_id	Business detail, e.g. a process-payment span

Real teams use both: auto for the basics, manual for key business steps.

2.5 OTLP and configuration

OTLP (OpenTelemetry Protocol) is the standard format + transport for telemetry: gRPC on port **4317**, HTTP on **4318**. Apps **push** OTLP. The SDK is configured with 0TEL_* environment variables, so no addresses live in the code:

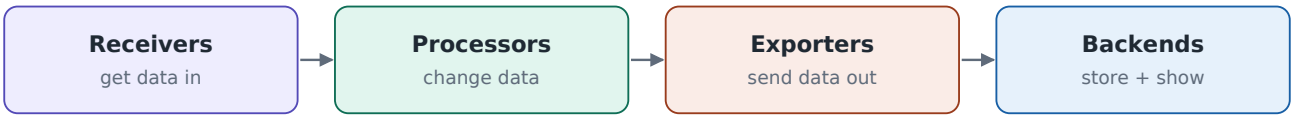
Variable	Meaning
OTEL_SERVICE_NAME	The app's name, shown in Jaeger and Grafana
OTEL_EXPORTER_OTLP_ENDPOINT	Where to send: http://otel-collector:4317
OTEL_EXPORTER_OTLP_PROTOCOL	How: grpc or http/protobuf
OTEL_TRACES / METRICS / LOGS_EXPORTER	Turn each signal on (otlp) or off (none)
OTEL_PYTHON_LOGGING_AUTO_INSTRUMENTATION_ENABLED	Capture Python logging and attach trace_id and span_id

2.6 Correlation

Every log line written during a request carries that request's **trace_id**. In Grafana the trace_id becomes a **link** that opens the exact trace in Jaeger: **log to trace in one click**.

3. The OpenTelemetry Collector

The Collector is a **middleman**: it receives telemetry from many sources, processes it, and sends it to one or more backends.



Nothing runs unless it is listed in service.pipelines.

Section	Simple meaning	Used in this lab
receivers	How data comes in	prometheus (pulls node-exporter), otlp (apps push on 4317/4318)
processors	What happens on the way	memory_limiter (safety), attributes/env (adds environment="lab"), batch
exporters	Where data goes out	prometheus (served on :8889), otlp/jaeger, otlphttp/loki, debug
service.pipelines	Switches it on, one pipeline per signal	metrics, traces, logs

Rules to remember

- A component key is `type[/name]`: `otlp/jaeger` means type otlp, named jaeger. The name lets you use one type twice.
- A component that is defined but **not listed in a pipeline** does nothing.
- Inside the Collector, all data uses **one common OTel format**. Receivers convert in, exporters convert out, so any input works with any output.
- Pull vs push exporters: the **prometheus** exporter opens a port and waits to be scraped; **otlp** and **otlphttp** exporters push data to the backend.

Listen vs connect

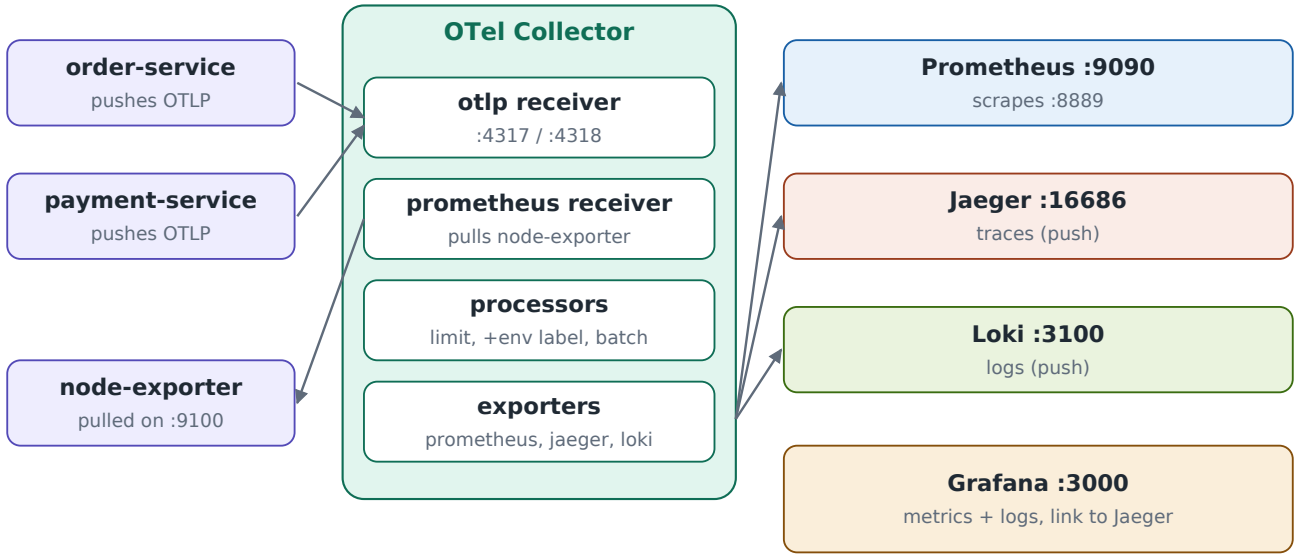
Address	Meaning
0.0.0.0:4317	Listen on all network interfaces of this container (receivers, the prometheus exporter)
jaeger:4317	Connect to the container named jaeger. Docker DNS turns the name into its current IP

Same port, no clash. 4317 is the standard OTLP port, so both the Collector and Jaeger listen on it. Each container has its own IP address, like flat 101 in two different buildings.

4. Tech stack: what each tool does

Tool	Role in this lab
Docker Compose	Runs all 8 containers on one network; they reach each other by service name
OTel Collector (contrib)	Receives, processes and routes all signals. Contrib adds extra components, e.g. the prometheus receiver/exporter
OTel Python SDK + auto-instrumentation	Produces telemetry from the apps with no code changes
Python Flask apps	order-service calls payment-service; payment fails about 20% of the time
node-exporter	Infrastructure metrics (CPU, memory, disk) in Prometheus format
Prometheus	Stores metrics; PromQL queries
Jaeger	Stores traces; UI to search, view and compare them
Loki	Stores logs; queried through Grafana with LogQL
Grafana	One place for metrics and logs, with a link from each log to its trace

5. Architecture, data flow and ports



Signal paths

Signal	Path
Metrics (infra)	node-exporter, pulled by the Collector, served on :8889, scraped by Prometheus, shown in Grafana
Metrics (app)	apps push OTLP to the Collector, served on :8889, scraped by Prometheus
Traces	apps push OTLP to the Collector, which pushes to Jaeger
Logs	apps push OTLP to the Collector, which pushes to Loki; viewed in Grafana

Ports

Container	Port	Purpose
node-exporter	9100	Serves node metrics at /metrics
otel-collector	4317 / 4318	OTLP receiver (gRPC / HTTP): apps push here
otel-collector	8889	Prometheus exporter: Prometheus scrapes here
jaeger	4317	Receives traces from the Collector (internal only)
jaeger	16686	Jaeger UI
loki	3100	Receives logs at /otlp and answers Grafana queries
prometheus	9090	Prometheus UI and query API
grafana	3000	Grafana UI
order-service / payment-service	5000 / 5001	The demo apps

6. How this helps in real production

Use case	How OpenTelemetry helps
Vendor freedom	Instrument once; switch Jaeger to Tempo, or Prometheus to Datadog, by changing only the Collector exporter
Faster debugging	Error log, then trace_id, then the full request path: which service failed and how long each step took
Latency hunting	Find requests slower than an SLO target (e.g. 400 ms) and see exactly which span is slow
One pipeline	Metrics, traces and logs go through one Collector with the same labels (environment, cluster)
Cost control	Processors drop noisy data, remove sensitive fields, or keep only errors and slow traces
Zero-code start	Auto-instrumentation gives useful telemetry from day one
Kubernetes, multi-cluster	Collectors per node or cluster forward to a central backend: one Grafana for many clusters

7. Lab: step by step

Files: `docker-compose.yaml`, `otel-collector-config.yaml`, `prometheus.yaml`, `apps/order.py`, `apps/payment.py`. Tool: Docker Desktop.

Step 1: start everything

```
docker compose up -d
docker compose ps
docker compose logs -f order-service
```

You will see: 8 containers Up. The order-service log ends with "Running on http://0.0.0.0:5000" after about 30 to 60 seconds (packages install at start).

Step 2: send traffic

```
for i in $(seq 1 30); do curl -s localhost:5000/order; echo; done
```

You will see: about 80% "status":"paid" and 20% "status":"failed".

Step 3: metrics, following the data through the Collector

```
curl -s localhost:9100/metrics | grep "^node_memory_MemAvailable"   # source
curl -s localhost:8889/metrics | grep "^node_memory_MemAvailable"   # after the Collector
```

You will see: the second line contains environment="lab", added by the processor. In Prometheus (localhost:9090) the otel-collector target is UP; query node_memory_MemAvailable_bytes{environment="lab"}. honor_labels: true keeps the original job="node-exporter" label.

Grafana (localhost:3000, admin / admin): add a Prometheus data source with URL http://prometheus:9090, then use Explore. App metrics pushed via OTLP appear as http_server_* metrics; list them with:

```
curl -s localhost:8889/metrics | grep -E "^http_(server|client)" | cut -d'{' -f1 | sort -u
```

Step 4: traces in Jaeger

Open localhost:16686, choose Service order-service, then Find Traces. Open one trace.

Try this	How
See one request	2 services, 3 spans: GET /order, GET (call), payment GET /pay
Only failed requests	Attributes: http.status_code=500
Only slow requests	Min Duration: 400ms
Compare two traces	Tick two results, then Compare Traces
Service map	System Architecture tab

Step 5: logs in Grafana (Loki)

Add a Loki data source with URL http://loki:3100. In Explore, choose loki and run one query at a time:

```
{service_name="payment-service"}
{service_name="payment-service"} |= "declined"
{service_name=~"order-service|payment-service"} | severity_text="ERROR"
```

You will see: each log line carries trace_id, span_id, severity_text, code_file_path, code_line_number and environment=lab.

Step 6: one-click log to trace

In the loki data source, under Derived fields, add:

Setting	Value
Name / Type / Label	TraceID / Label / trace_id
URL	http://localhost:16686/trace/${__value.raw}
Open in new tab	On

You will see: after Save & test, run a fresh query and expand a log line. Under Links, click TraceID: Jaeger opens the exact request that wrote the log.

8. Production vs this lab

This lab	Real production
Docker Compose on a laptop	Kubernetes: the OpenTelemetry Operator manages Collectors and auto-instrumentation
One Collector	Agent Collectors per node (DaemonSet) + a gateway Collector per cluster
Jaeger	Grafana Tempo (native Grafana integration) or a SaaS backend
Prometheus	Mimir or Thanos for long-term, multi-cluster metrics
Data in memory, no auth	Persistent storage, TLS, authentication
All traces kept	Tail sampling: keep errors and slow traces, sample the rest
environment="lab" label	cluster, region and team labels for multi-cluster views

Clean-up

```
docker compose down
docker image prune -a     # optional: remove unused images
```

9. Interview questions: short answers

What is OpenTelemetry?

A vendor-neutral CNCF standard, with SDKs and a Collector, for producing and sending metrics, traces and logs to any backend.

Prometheus vs OpenTelemetry?

Prometheus stores and queries metrics and mostly pulls. OpenTelemetry produces and ships all three signals and does not store anything. They are often used together.

What are the parts of the Collector?

Receivers (data in), processors (change data), exporters (data out), and service.pipelines that wire them together per signal.

What is OTLP?

The OpenTelemetry Protocol: the standard way to send telemetry, over gRPC (4317) or HTTP (4318).

Trace vs span?

A span is one unit of work with timing and attributes. A trace is the tree of spans for one request, sharing one trace_id.

How does a trace cross two services?

Context propagation: the caller adds a traceparent header with the trace_id; the callee reads it and continues the same trace.

Auto vs manual instrumentation?

Auto hooks frameworks with no code changes (HTTP spans, metrics, logs). Manual adds business-level spans and metrics in code.

Why use a Collector instead of exporting from the app straight to the backend?

Apps only need one OTLP address. The Collector adds batching, retries, filtering, enrichment and routing, and lets you change backends without touching the apps.

How are logs linked to traces?

The SDK attaches the active trace_id and span_id to each log record, so a tool like Grafana can jump from a log line to its trace.

What is tail sampling?

A Collector processor that decides after a trace is complete whether to keep it, for example keeping all errors and slow traces and only a sample of the rest, to cut cost.

10. Official references

#	Reference	Link
1	OpenTelemetry documentation	opentelemetry.io/docs/
2	OpenTelemetry Collector configuration	opentelemetry.io/docs/collector/configuration/
3	Python zero-code instrumentation	opentelemetry.io/docs/zero-code/python/
4	Free course: Getting Started with OpenTelemetry (LFS148)	training.linuxfoundation.org/training/getting-started-with-opentelemetry-lfs148/
5	Jaeger documentation	www.jaegertracing.io/docs/
6	Loki: ingesting OpenTelemetry logs	grafana.com/docs/loki/latest/send-data/otel/